

Le due librerie

Che cosa possediamo, che problemi risolve al posto nostro, e che cosa non farà mai

In dieci righe

Il pagAmico parla un protocollo TCP proprio: apri un socket, gli scrivi dentro stringhe corte come `IN001550`, lui risponde. Il problema è che quel protocollo, visto da un'applicazione, non è una conversazione ordinata: non c'è un terminatore che dica dove finisce una risposta, a un comando possono seguire tre o quattro risposte diverse, alcune sono JSON e altre testo semplice, alcuni comandi non rispondono affatto, gli importi in andata sono in centesimi e al ritorno in euro, e i nomi dei campi cambiano fra una versione di firmware e l'altra.

La libreria si mette in mezzo. Tiene aperta la connessione, ritaglia i messaggi dal flusso di byte, capisce quale delle risposte ricevute è davvero l'esito del comando che hai mandato, converte le unità di misura, normalizza i nomi dei campi e traduce i codici d'errore in frasi italiane. Sopra a questo espone cinquantotto metodi dove tu passi un importo in euro e ti torna un oggetto tipizzato.

In una frase: trasforma un protocollo seriale travestito da TCP in un normale SDK. Chi la usa non deve più sapere niente di socket, di buffer e di refusi del manuale.

1. I cinque problemi veri che risolve

Non sono difficoltà teoriche: sono le cose che si scoprono solo provando, e che costano una giornata ciascuna quando ci si sbatte contro da soli.

1.1 Dove finisce un messaggio

Il problema. Il TCP è un flusso di byte, non di messaggi. Il pagAmico non usa un terminatore unico: le risposte normali sono oggetti JSON senza delimitatore finale, le liste movimenti finiscono con un marcatore proprio, e certe risposte sono testo nudo senza niente in fondo — `CMD ERROR`, `ER BUSY`, `BT1` per un bottone premuto, `EX` per una lista chiusa, oppure il contenuto di un codice a barre appena letto.

Che cosa succede senza. Due casi, entrambi visti solo provando. Se tratti ogni lettura dal socket come un messaggio, prima o poi due risposte arrivano insieme e te ne perdi una — oppure una risposta arriva spezzata in due e provi a interpretare mezzo JSON. Se invece aspetti un fine-riga per chiudere il messaggio, resti bloccato per sempre sulle risposte testuali, che il fine-riga non ce l'hanno.

Come è risolto. Il parser guarda il primo carattere e decide. Se è una graffa conta le graffe **ignorando quelle dentro le stringhe** — serve perché lo scontrino POS contiene graffe, virgolette e barre rovesciate. Se non è una graffa chiude il messaggio su tre condizioni alternative: sulla graffa se subito dopo comincia un JSON, su un fine-riga, oppure **dopo 150 millisecondi di silenzio**. C'è anche una protezione anti-stallo: un JSON che resta incompleto per dieci secondi viene consegnato comunque come testo grezzo, invece di bloccare tutto.

1.2 Quale delle risposte è l'esito

Il problema. Non è domanda-e-risposta. Mandi "incassa 15,50" e ricevi, nell'ordine: un ok che significa "ho accettato il comando", poi un messaggio ripetuto a ogni moneta infilata, e alla fine l'esito vero. Fra l'inizio e la fine possono passare minuti.

Che cosa succede senza. L'applicazione prende il primo ok per l'esito, dichiara l'incasso concluso e va avanti mentre il cliente sta ancora mettendo monete. E chi programma non se ne accorge in ufficio, perché sul simulatore la sequenza è istantanea.

Come è risolto. Ogni comando registra un'attesa con un predicato che dice quale messaggio la chiude; tutti gli altri vengono girati all'applicazione come avanzamento, così si può mostrare al cliente l'importo che sale.

Due casi speciali che si scoprono solo sul campo:

- **Il comando che chiude un incasso trattenendo il parziale risponde due volte**, almeno sul simulatore: quanti messaggi arrivino sulla macchina è la prima domanda della telefonata. Il primo è l'accettazione, il secondo porta l'importo trattenuto. Anche l'accettazione porta `committedAmount`, valorizzato a zero: fino al 10 settembre la libreria chiudeva lì e il chiamante riceveva trattenuto 0,00 (difetto D1 in `md/memoria-claude/esito-risposta-payprint.md`). Ora chiude sul messaggio con `errorCode` diverso da `OK` — `OK` nell'accettazione, vuoto nell'esito — e il trattenuto si legge in `collectedAmount`. C'è anche un avviso nel codice: dopo quel comando possono ancora arrivare messaggi di avanzamento, perché l'hopper legge più in fretta di quanto il comando viaggi.
- **Annullare un incasso non significa smettere di ascoltare.** I soldi sono già dentro la macchina: mollare l'attesa significa non sapere quanto va restituito.
- **Durante un incasso un testo non è l'esito.** Prima dell'ok di accettazione un testo vuol dire che la macchina ha rifiutato l'incasso (`BUSY`, `CMD ERROR`); dopo, è la risposta a qualcos'altro e non tocca l'incasso. Il predicato lavora quindi in due fasi. E ciò che nessuna attesa riconosce non si butta: va all'evento dei **frame orfani**, perché può portare importi.

1.3 Due comandi di fila, e il secondo sparisce

Il problema. Senza terminatore — ed è così che le librerie inviavano fino al 14 settembre — il pagAmico legge il buffer del socket e lo interpreta come **un solo** comando. Se ne scrivi due a raffica finiscono nello stesso segmento TCP e il secondo viene ignorato — senza errore, senza risposta, senza niente.

Che cosa succede senza. Il caso classico: chiudi la finestra sul display e subito dopo lanci l'incasso. L'incasso non parte, non arriva nessun messaggio, l'applicazione va in timeout dopo cinque minuti e non c'è modo di capire perché. È esattamente il tipo di difetto che si riproduce una volta su tre.

Come è risolto. Ogni invio misura la distanza dall'invio precedente e, se serve, aspetta prima di scrivere. Il valore di default è 80 millisecondi. Il commento nel codice la dichiara necessaria: *verificato sul simulatore, senza pausa il comando si perde, con 30 ms passa; il default tiene un margine.*

Questo era il difetto numero uno del progetto, e la prima domanda a PayPrint (1.1 della mail, col terminatore alla 1.2). La risposta dell'11 settembre: **nessuna pausa serve, basta che ogni comando termini con CR o CR+LF**; il simulatore «probabilmente ha qualche difficoltà». Dal 14 settembre il terminatore di default è **CR**, nelle due librerie e nei due banchi, anche se il manuale 2.33 non lo nomina mai. Sul simulatore la raffica regge ormai con e senza CR (md/verbali/prova-terminatore-cr-2026-09-14.md). Gli 80 ms, **misura empirica sul simulatore** e non minimo garantito dal firmware, restano come rete di sicurezza finché la raffica con CR non regge su una macchina vera: l'ordine di lavoro è in md/memoria-claude/esito-risposta-payprint.md.

1.4 Centesimi in andata, euro al ritorno

Il problema. Nei comandi gli importi sono centesimi, come stringa a lunghezza fissa con gli zeri davanti; nelle risposte sono euro con la virgola. E la lunghezza del campo cambia da comando a comando: l'incasso ne vuole sei, l'erogazione dieci.

Che cosa succede senza. Il fattore cento sbagliato in un verso o nell'altro: incassi 0,15 invece di 15,00, oppure la macchina risponde con un errore di lunghezza e nessuno capisce che mancava uno zero. È un errore che passa la revisione del codice perché il codice *sembra giusto*.

Come è risolto. Un solo punto di conversione, con arrotondamento commerciale, e un solo formattatore che impone la lunghezza e **rifiuta** valori negativi o troppo lunghi con un'eccezione parlante. Da lì in poi ogni comando è una riga sola e non ci si può sbagliare. In lettura, tutti gli importi tornano come decimali in euro — mai in virgola mobile.

1.5 I cassetti delle banconote non sono in ordine

Il problema. La risposta di stato contiene una matrice di cassetti, e i manuali non dicono che cosa c'è in ciascuna posizione. Peggio: **i cassetti non sono ordinati per taglio, e lo stesso taglio può comparire più volte**.

Che cosa succede senza. Chi deduce il taglio dalla posizione nell'array legge numeri sbagliati — e nel momento peggiore, cioè quando deve decidere se la macchina può dare il resto.

Come è risolto. Le mappe posizionali sono state ricavate dalla guida del Dev Kit, che è l'unica fonte che le dà. Il taglio si legge sempre dal contenuto, mai dalla posizione, e la libreria espone direttamente le giacenze **sommate per taglio**, così chi la usa non tocca mai l'array grezzo.

2. La mappa della superficie pubblica

Cinquantotto metodi pubblici, raggruppati per famiglia. Non serve conoscerli: serve sapere che esistono.

Famiglia	Metodi	A che serve
Incasso	6	prendere i soldi dal cliente: contanti, POS, automatico, con timeout, più annulla e chiusura-trattenendo
Erogazione	3	dare i soldi: un importo, oppure banconote e monete contate
Manutenzione	10	quello che fa l'operatore, non il cliente: svuotare, azzerare, soglie, quali tagli si accettano
Ricariche	4	sessioni di caricamento del contante
Stato	3	fondi, scorte, e la stringa che dice se serve l'operatore
POS	5	il terminale carte integrato: ultima transazione, totali, chiusura giornaliera
Movimenti	2	il pagAmico tiene un database interno; qui lo si interroga
Display	11	testo, finestre di messaggio, input da tastiera, lettura codici, liste scorrevoli
Immagini	3	logo permanente e immagine temporanea
Stampa	3	più un costruttore di scontrini a catena
Connessione e invio grezzo	8	aprire, chiudere, scrivere direttamente sul socket, attendere un messaggio senza inviare
Errori	—	non metodi ma vocabolario: traduce i codici in italiano e smonta la stringa di stato

Un dettaglio che conta più di quanto sembri. La famiglia degli errori sa dire, leggendo una stringa di quattro cifre, se la macchina è sottoscorta di monete, se ha troppe monete, se il cassetto di scarico è pieno — **e se l'importo richiesto è componibile o no.** È esattamente il dato che servirebbe per chiudere il buco del resto non erogato in Giano.

3. Quello che devi sapere e che non è scritto nella firma

Questa è la sezione che vale la lettura. Sono i vincoli che un'applicazione scopre tardi.

Sulla connessione

- **Il pagAmico è il server:** si apre *una* connessione e si tiene aperta. Non è connetti-chiedi-chiudi.
- **Non c'è riconnessione automatica.** Alla caduta, in entrambe le librerie, **tutte le attese in corso falliscono subito** con `PagAmicoConnectionLostException`, che dice anche se un incasso era già accettato (`MaybeCollecting`): in quel caso la macchina potrebbe stare ancora incassando. Fino al 10 settembre in Kotlin l'attesa restava sospesa fino al proprio timeout; era un prerequisito per togliere il timeout dell'incasso (`md/memoria-claude/esito-risposta-payprint.md`, D2), ed è fatto. Rifare la connessione è compito dell'applicazione.
- La porta di fabbrica è la 9100, ma il manuale stesso consiglia di cambiarla: è la porta di stampa più usata al mondo.
- Gli eventi arrivano dal **thread di ricezione**. In un'applicazione a finestre vanno portati sul thread della UI prima di toccare qualsiasi cosa a video.

Sui comandi

- **A incasso aperto partono solo AN e CM.** Il fornitore ha chiarito che durante **IN** la macchina accetta solo quei due. Dall'11 settembre la libreria lo fa rispettare: finché `IsCollecting` è vero, **ogni altro invio lancia subito** `PagAmicoCollectionOpenException` senza trasmettere nulla — display, immagini, stampa, ST compresi — invece di accodarsi dietro l'**IN** o di partire e provocare un **BUSY**. **CM** e **AN** passano per una via laterale fuori dal turno unico, dopo l'ok di accettazione, e **uno solo per incasso**: il secondo lancia, un **CM** rifiutato libera il posto. Fino al 10 settembre **CM** si accodava dietro l'**IN** e un display partiva davvero.
- **L'esito di un rifiuto dice se sono entrati soldi.** Un incasso rifiutato prima dell'ok lancia `PagAmicoRejectedException` (o `PagAmicoBusyException` se la macchina è impegnata): **non ha incassato nulla**. Una caduta lancia `PagAmicoConnectionLostException`: forse sta incassando. Fino al 10 settembre le tre situazioni uscivano con la stessa eccezione generica, e un testo qualsiasi chiudeva l'attesa dell'incasso (difetto D3 in `md/memoria-claude/esito-risposta-payprint.md`). Sui messaggi davvero spontanei — barcode, BT1, EX — la risposta del fornitore tace; se ne arrivassero durante un incasso, andrebbero ai frame orfani.
- **Anche i comandi semplici attendono la propria risposta.** Il proprio codice (ST su ST, SM su SM), un OK, oppure un errore: è quello che il simulatore manda a ogni comando del collaudo. Un altro messaggio arrivato nel frattempo va ai frame orfani. Fino all'11 settembre un ST prendeva per risposta il primo frame che arrivava, e sul simulatore un CM in ritardo ha fatto proprio questo; nel collaudo successivo lo stesso CM è arrivato di nuovo ed è finito, giustamente, fra gli orfani. Unica eccezione **LO**, che rimanda l'ultimo JSON qualunque sia.
- Finché resta la pausa (vedi 1.3), ogni invio può bloccare fino a 80 millisecondi. In un ciclo di venti comandi sono quasi due secondi.
- **Il timeout di transazione è cinque minuti, e per l'incasso il fornitore dice di non metterlo:** dopo **IN** la macchina non ne ha. Se scade, la libreria smette di attendere senza mandare **AN**: l'incasso **resta aperto** sulla macchina, il cliente può continuare a inserire denaro, e la libreria non lo sa e non lo compensa (difetto D2 in `md/memoria-claude/esito-risposta-payprint.md`). Toglierlo non è una riga: serviva il keepalive TCP regolato, che dal 14 settembre c'è (10 s, 2 s, 5 sonde: una caduta si vede in circa 20 s), e serve ancora la prova del cavo staccato su una macchina vera.

Sui parametri

- Quattro metodi vogliono **esattamente sei elementi**, e l'ordine è posizionale e non appare nella firma: monete da 0,05 a 2,00 e banconote da 5 a 200.
- La password di erogazione ha un massimo di diciannove caratteri; gli importi negativi sono rifiutati; l'incasso si ferma a **9.999,99 euro** perché ha sei cifre di centesimi, mentre l'erogazione arriva a dieci cifre. Per i contanti PayPrint conferma il tetto; per il POS dice «maggiore», ma po ha sei cifre fisse (manuale p. 17): il punto resta aperto.
- Nei testi del display **la barra verticale è il separatore del protocollo** e viene sostituita con una barra normale. Un testo che la contiene non arriva identico.

4. La libreria gemella in Kotlin

Esiste per una ragione pratica — la parte Android — e per una che vale altrettanto: **le due si controllano a vicenda**.

Condividono tutto tranne il linguaggio: stessi file, stessi nomi, stesso ordine, e soprattutto le stesse decisioni difficili — il conteggio delle graffe, la normalizzazione dei nomi dei campi, la pausa di 80 millisecondi, la conversione degli importi.

Le differenze sono imposte dalla piattaforma, non scelte di gusto:

C#	Kotlin
Task + token di cancellazione	funzioni sospese + cancellazione della coroutine
eventi	flussi osservabili
decimal	BigDecimal — e quindi i confronti non si scrivono con l'uguale
compila per tre piattaforme	una sola, JVM 17
test e collaudo in progetti separati	dentro la libreria, per avere una sola configurazione di avvio

La divergenza più insidiosa fra le due, ma non è l'unica. Sull'annullo di un incasso le due librerie **non fanno la stessa cosa**. In C# l'attesa prosegue e il chiamante riceve l'esito, quindi sa quanto è stato restituito. In Kotlin l'annullo parte davvero, ma la cancellazione arriva subito al chiamante: **l'esito arriva a onOrphanFrame**, perché l'incasso vive nello scope del client e l'attesa continua. Chi scriverà il flusso di pagamento sulla Kotlin deve ascoltare onOrphanFrame, o annullare con `cancelOperation()`, che restituisce l'esito. Fino al 10 settembre, in Kotlin, quell'esito andava perso. L'altra divergenza ancora aperta è il **registro su file** — che in C# si esclude fra processi con un mutex di sistema, mentre in Kotlin sincronizza solo dentro il processo. Il rischio è minore di quanto si pensava: il test con due logger sullo stesso file, da due thread, **passa anche in Kotlin senza mutex**, perché Java apre il file in modalità di accodamento del sistema operativo e ogni riga resta intera; in C# `FileMode.Append` fissa invece la posizione all'apertura, e senza mutex le righe si sovrascrivono. Fra due processi Kotlin veri la prova non è stata fatta. La terza divergenza, la **caduta della connessione**, è stata allineata l'11 settembre. Sul mutex C# l'11 settembre è emerso anche un difetto, corretto: il suo nome si calcolava una volta sola, sul file del primo giorno, e dopo mezzanotte un processo avviato ieri e uno avviato oggi scrivevano lo stesso file con due mutex diversi. Ora il nome segue il file del giorno, e un test lo verifica.

La parità è disciplina, non un meccanismo

Va detto chiaramente perché è il punto più fragile del lavoro. **Non esiste nessun controllo automatico** che verifichi che le due librerie siano allineate: nessuna integrazione continua, nessuno script di confronto, nessun generatore che produca i due sorgenti da un'unica descrizione.

Detto questo, l'allineamento è costruito in modo da essere verificabile, e questo cambia molto:

- Le due suite di test sono **specchiate**: centosessantotto asserzioni per parte, stesse nove sezioni, stesso ordine, stessi nomi (a parte due, preesistenti, con la maiuscola idiomatica). Le quattro sezioni aggiunte l'11 settembre fanno parlare il client vero con un finto `pagAmico` su 127.0.0.1 (sequenze di incasso, comandi semplici e invio) e provano il registro su file e il vocabolario degli errori.
- E soprattutto: **i test non confrontano le due librerie fra loro, confrontano ciascuna con i manuali**. Se entrambe verificano che il comando prodotto sia la stringa letterale stampata sul manuale, allora sono allineate *per costruzione* — non perché qualcuno le ha confrontate, ma perché misurano lo stesso metro esterno.
- Anche i nomi dei passi del collaudo coincidono uno per uno.

Il rischio residuo è concreto: se domani qualcuno aggiunge un comando da una parte sola, nessun test fallisce. L'unico indicatore è il conteggio stampato a fine esecuzione, da confrontare a occhio con quello dell'altra libreria.

Se si vuole trasformare la disciplina in un controllo, la cosa più economica è uno script che estragga i nomi delle asserzioni dai due file di test e i nomi dei passi dai due collaudi, li confronti, e si lamenti se un nome esiste da una parte sola. È mezza giornata, e la parte difficile — usare gli stessi nomi — è già fatta.

5. Dove sta davvero la difficoltà

File	Righe	Complessità
PagAmicoClient	1.229	alta, ma concentrata
PagAmicoResponse	409	alta
PagAmicoErrors	334	bassa: sono dizionari, più le eccezioni
PagAmicoPrint	273	bassa
PagAmicoDisplay	270	bassa
PagAmicoCommands	257	bassa, ma delicata
PagAmicoFileLogger	235	media, e completamente isolata
PagAmicoFrameParser	180	la più densa per riga
PagAmicoFrame	70	nulla
Compatibility	16	nulla

Non sta nei file grossi. Su 3.273 righe (2.848 prima delle correzioni dell'11 settembre), **quelle rischiose sono circa 800**, concentrate in tre punti:

1. **Il parser dei messaggi, per intero.** È il file più piccolo dopo i due banali ed è quello dove un errore si paga di più: se sbagli a ritagliare i messaggi, tutto il resto della libreria riceve spazzatura.
2. **Il cuore del client**, circa seicento righe commenti compresi: invio con distanza minima e blocco a incasso aperto, attese a tre esiti, loop di ricezione, smistamento con i frame orfani, e la macchina a stati dell'incasso — accettazione, un solo comando di chiusura, via laterale per AN e CM. È il pezzo concorrente — una serratura, due semafori, un loop in background che deve sopravvivere alle eccezioni — ed è quello coperto dai trentacinque test di sequenza. Le altre seicento righe del file, quelle dei cinquantotto metodi, sono quasi tutte ripetitive.
3. **Le mappe taglio-quantità e la normalizzazione delle chiavi.** Poche righe che incapsulano tutte le stranezze del firmware.

Il registro su file merita una nota a parte: le sue duecentotrentacinque righe sono complesse ma **completamente isolate**. Il motivo per cui è fatto così è scritto nel codice: due processi che scrivono lo stesso file in append si sovrascrivono a vicenda e metà delle righe sparisce senza errori — inaccettabile per un file che deve valere come prova di un incasso.

6. Che cosa non fa, e non farà mai

Non gestisce lo stato della transazione, oltre il minimo. Dall'11 settembre sa se c'è un incasso aperto (IsCollecting) e lo usa per bloccare gli invii; ma non tiene un giornale, non ha un database. Se l'applicazione va in timeout o si riavvia a metà incasso, la libreria non ricostruisce niente: al massimo consegna all'evento dei frame orfani quello che arriva dopo.

È la scelta più importante da conoscere, perché ricade interamente su chi integra: la riconciliazione contabile è dell'applicazione. In Giano, oggi, quella riconciliazione **non esiste** — vedi il documento 4.

Non riconnette da sola. Nessun tentativo automatico, nessun battito cardiaco applicativo. Alla caduta segnala e si ferma.

Non è fiscale. La stampa è testo, codici a barre e QR: nessuno scontrino fiscale, nessun registratore telematico.

Non gestisce più macchine. Un'istanza è un indirizzo. Per due casse servono due istanze.

Non decide al posto tuo se un'operazione ha senso. Non verifica prima di erogare che ci siano banconote a sufficienza: manda il comando e traduce l'errore che torna. Le uniche validazioni sono formali — lunghezze, intervalli, conteggi.

Non logga da sola. Il registro su file esiste ma va agganciato esplicitamente. Una libreria che scrive file senza che glielo si chieda sarebbe sbagliata — ma vuol dire che se nessuno lo aggancia, quando una cassa contesta un incasso non c'è nessuna prova di che cosa è stato chiesto alla macchina.

Protegge l'incasso, non tutto il resto. Serializza i comandi che attendono risposta e, a incasso aperto, rifiuta ogni invio che non sia AN o CM. Fuori da un incasso, un comando senza risposta mandato mentre un altro attende la sua può ancora confondere le risposte.

7. I punti ancora incerti dentro la libreria

Non sono difetti: sono posti dove il manuale tace o si contraddice, e dove è stata presa una decisione motivata in attesa di conferma.

Punto	Che cosa si è fatto
Formato del pacchetto immagine	il manuale a testo e il suo stesso esempio Python danno due layout diversi: implementati entrambi , selezionabili, con quello descritto a testo come predefinito
Comando di input sul display	non documentato in nessun manuale: implementato nella forma a sette campi, con la forma a dodici trovata nel Dev Kit provata solo dal collaudo
Tipi di codice a barre	il manuale di stampa si contraddice fra pagina 3 e pagina 5: scelta la versione che concorda col Dev Kit, che è anche l'emulatore
Riavvio del POS	la tabella dice una sigla, il dettaglio dello stesso paragrafo un'altra: scelta la prima
Sequenze di stampa dirette	il manuale usa tre prefissi diversi in tre punti

Il testo completo di queste cinque domande sta nella **mail a PayPrint** (md/fornitore/mail-payprint-domande-protocollo.md). Nel primo messaggio ne partono però solo due — la distanza minima fra due comandi e il terminatore, domande 1.1 e 1.2 — e l'11 settembre hanno avuto risposta: sulla macchina i comandi vanno chiusi con CR o CR+LF, e col terminatore la pausa non serve; sul simulatore il CR è stato provato il 14 settembre ed è diventato il default (md/verbalì/prova-terminatore-cr-2026-09-14.md). Le due che riguardano display e immagini — il formato di DI e l'incapsulamento di SF/SI — all'innesto in Giano non servono e sono state messe da parte per un secondo giro (sezione A della mail); i tipi di codice a barre stanno con le altre note ai manuali, in sezione D. Le sei domande nate dall'innesto, che sono di natura diversa, stanno nel documento 5.